

EX200

Guia de Estudos — Dia 06/05

Scripts Shell · grep · sed · find · Variáveis de Ambiente

B1

Estrutura Base

B2

Argumentos

B3

Loop for

B4

for+\$@+if

B5

while+arquivo

B6

read+aritmética

B7

case+tr

B8

find SUID

B9

grep·sed·redir

B10

Variáveis Amb.

REFERÊNCIA RÁPIDA — VARIÁVEIS ESPECIAIS DO SHELL

\$1 \$2 \$3

argumentos posicionais (1°, 2°, 3°)

todos os argumentos como \$*

número de argumentos

\$()

substituição de comando

\$(())

aritmética inteira

-gt -lt -eq

comparadores numéricos: maior, menor

B1

Estrutura Base de Todo Script

shebang + variáveis + argumentos + chmod +x — SEMPRE

Todo script no EX200 segue exatamente a mesma estrutura. Internalize como um ritual: shebang → lógica → chmod +x → testar.

```
#!/bin/bash
# Linha 1: SEMPRE shebang #!/bin/bash

# Variáveis
NOME="valor"
RESULTADO=$(comando)      # capturar saída de comando
SOMA=$((NUM1 + NUM2))     # aritmética inteira

# Argumentos posicionais
$1 $2 $3                  # primeiro, segundo, terceiro argumento
"$@"                      # TODOS os argumentos como lista
$#                         # número de argumentos

# Criar, tornar executável e testar
vim /usr/local/bin/meu_script.sh
chmod +x /usr/local/bin/meu_script.sh
/usr/local/bin/meu_script.sh arg1 arg2
```

■ **chmod +x é ELIMINATÓRIO.** Sem ele o avaliador não consegue executar o script para corrigir — perde o ponto mesmo

■ **#!/bin/bash sem o shebang** o sistema tenta executar como binário e falha. Sempre na linha 1, sem espaço antes do #.

B2

Argumentos — \$1, \$2, \$@

flipargs.sh · checkfile.sh · \$1=1° arg · \$2=2° arg · aspas em \$1

S1·Q11 — flipargs.sh: inverter dois argumentos

```
#!/bin/bash
echo "$2 $1"

chmod +x /usr/local/bin/flipargs.sh

# Testar
/usr/local/bin/flipargs.sh hello world
# Output: world hello
```

■ \$1 = primeiro argumento, \$2 = segundo. Inverter a ordem no echo é tudo que precisamos.

S1·Q30 — checkfile.sh: verificar se arquivo existe

```
#!/bin/bash
if [ -f "$1" ]; then
    echo "File exists."
else
    echo "File missing."
fi

chmod +x /usr/local/bin/checkfile.sh

# Testar
/usr/local/bin/checkfile.sh /etc/passwd
# Output: File exists.
/usr/local/bin/checkfile.sh /etc/naoexiste
# Output: File missing.
```

Flags de teste — Referência:

Flag	Significado	Exemplo
-f	É arquivo regular	[-f "/etc/passwd"]
-d	É diretório	[-d "/tmp"]
-e	Existe (arquivo ou dir)	[-e "/caminho"]
-z	String vazia	[-z "\$VAR"]
-n	String não vazia	[-n "\$VAR"]

- Aspas em "\$1" evitam erros com nomes de arquivo com espaços. Sempre cite variáveis dentro de [].

B3

Loop for com Lista Fixa

user_audit.sh · user-list.sh · `$(id -u $USER)` · `$()` substituição

S1-Q24 — user_audit.sh: imprimir nome e UID de cada usuário

```
#!/bin/bash
for USER in root adm ftp
do
    USERID=$(id -u $USER)
    echo "User $USER has ID $USERID"
done

chmod +x /usr/local/bin/user_audit.sh

# Output:
# User root has ID 0
# User adm has ID 3
# User ftp has ID 14
```

■ `$(id -u $USER)` retorna só o número UID. `$(id $USER)` retorna `uid=0(root) gid=...` — texto completo que não é o pedido.

■ `$()` é substituição de comando: executa o comando em subshell e captura a saída. Forma moderna de backticks — mais

S3-Q26 — user-list.sh: imprimir mensagem para cada usuário

```
#!/bin/bash
for USER in root bin daemon
do
    echo "Processing user: $USER"
done

chmod +x /usr/local/bin/user-list.sh

# Output:
# Processing user: root
# Processing user: bin
# Processing user: daemon
```

Estrutura do for com lista — template mental:



```
for VARIABEL in item1 item2 item3
do
    # corpo do loop – $VARIABEL tem o item atual
done
```

B4

Loop for com \$@ + if — Somar Positivos

DESTAQUE: Script mais complexo do exame — Sim5·Q13 e Sim6·Q16

Este padrão aparece em dois simulados. É o script mais complexo do EX200. Combina: \$@ para número ilimitado de argumentos + for para iterar + if para filtrar + \$(()) para somar. Pratique até memorizar.

S5·Q13 e S6·Q16 — sum.sh: somar apenas argumentos maiores que 0

```
#!/bin/bash
TOTAL=0
for NUM in "$@"
do
    if [ "$NUM" -gt 0 ]; then
        TOTAL=$((TOTAL + NUM))
    fi
done
echo "The sum is $TOTAL"

chmod +x /sum.sh

# Testar
/sum.sh 10 20 -5 30
# Output: The sum is 60 (10+20+30, ignora -5)
/sum.sh 10 20 -5
# Output: The sum is 30
```

Cada elemento deste script explicado:

- **TOTAL=0** — Inicializar acumulador ANTES do loop — sem isso a aritmética pode falhar
- **"\$@"** — Expande para todos os argumentos como itens separados — com aspas preserva espaços
- **-gt 0** — greater than 0 — filtra negativos e zero
- **\$((TOTAL + NUM))** — Aritmética inteira — resultado atribuído à mesma variável

Tabela de comparadores numéricos:

Operador	Significado	Exemplo
-gt	maior que	["\$N" -gt 0]
-lt	menor que	["\$N" -lt 10]
-eq	igual	["\$A" -eq "\$B"]
-ne	diferente	["\$A" -ne 0]
-ge	maior ou igual	["\$N" -ge 1]
-le	menor ou igual	["\$N" -le 100]

B5

Loop while — Ler Arquivo Linha por Linha

while read VAR1 VAR2 + done < arquivo — S4·Q22

S4·Q22 — process_lines.sh: processar arquivo com DATA NOME

```
# Primeiro criar o arquivo de teste
cat > /root/dates.txt << EOF
2000-05-01 Sam
2002-06-15 Sue
2004-12-29 Sarah
2005-05-10 Mike
EOF

#!/bin/bash
while read DATE NAME
do
    echo "Name: $NAME was born on $DATE"
done < /root/dates.txt

chmod +x /usr/local/bin/process_lines.sh

# Output:
# Name: Sam was born on 2000-05-01
# Name: Sue was born on 2002-06-15
# Name: Sarah was born on 2004-12-29
# Name: Mike was born on 2005-05-10
```

■ `done < /root/dates.txt` redireciona o arquivo como entrada do loop. `while read VAR1 VAR2` divide cada linha pelo espaço

■ **NÃO** use `cat arquivo | while read` (pipe cria subshell — variáveis modificadas dentro não ficam visíveis fora). Use redireção

B6

read Interativo + Aritmética \$(())

echo + read + \$(()) — S3-Q12 — distinção: interativo vs argumentos

S3-Q12 — sum.sh interativo: pedir dois números ao usuário

```
#!/bin/bash
echo "Enter the first number"
read num1
echo "Enter the second number"
read num2
sum=$((num1 + num2))
echo "The result of addition=$sum"

chmod +x /usr/local/bin/sum.sh

# Execução interativa:
/usr/local/bin/sum.sh
# Enter the first number
# 5
# Enter the second number
# 3
# The result of addition=8
```

Operações com \$(()):

Expressão	Operação
<code>\$((A + B))</code>	Soma
<code>\$((A - B))</code>	Subtração
<code>\$((A * B))</code>	Multiplicação
<code>\$((A / B))</code>	Divisão inteira
<code>\$((A % B))</code>	Módulo (resto)

■ **LEIA o enunciado:** 'pede ao usuário' → read interativo. 'aceita argumentos' → \$1 \$2. Confundir os dois = script errado.

B7**case + tr — Case-Insensitive**

yes-no.sh · tr '[:upper:]' '[:lower:]' · *) default · esac — S2·Q9

S2·Q9 — yes-no.sh: resposta diferente para yes/no/outros

```
#!/bin/bash
# Converter para minúsculo para comparação case-insensitive
INPUT=$(echo "$1" | tr '[:upper:]' '[:lower:]')

case "$INPUT" in
    yes)
        echo "That is nice"
        ;;
    no)
        echo "I am sorry"
        ;;
    *)
        echo "Unknown argument"
        ;;
esac

chmod +x /usr/local/bin/yes-no.sh

# Testar
/usr/local/bin/yes-no.sh yes      # That is nice
/usr/local/bin/yes-no.sh YES     # That is nice (case-insensitive!)
/usr/local/bin/yes-no.sh no      # I am sorry
/usr/local/bin/yes-no.sh maybe   # Unknown argument
```

Anatomia do case:

- **tr '[:upper:]' '[:lower:]'** — Converte TUDO para minúsculas antes de comparar
- **case "\$INPUT" in** — Abre o case — variável entre aspas
- **yes)** — Opção que casa exatamente com 'yes'
- **;;** — Termina cada bloco — obrigatório, como break
- ***)** — Default — equivalente ao else — casa qualquer coisa
- **esac** — Fecha o case — 'case' ao contrário

■ **case é mais limpo que if/elif/else para múltiplas opções do mesmo valor. Esquecer ;; ou esac causa erro de sintaxe.**

B8

Scripts com find — SUID e Auditoria

-perm -4000 · -perm /u=s · -size -10M · -exec cp -a {} \;

S3-Q28 — find_suid.sh: listar arquivos SUID em /usr/bin



```
#!/bin/bash
find /usr/bin -user root -perm -4000

chmod +x /root/find_suid.sh
# Output: /usr/bin/sudo, /usr/bin/passwd, etc.
```

■ perm -4000: o hífen significa 'pelo menos este bit ativo'. Sem hífen (-perm 4000) exige permissão EXATAMENTE 4000 —

S4-Q27 — audit_suid.sh: copiar arquivos SUID < 10MiB para /root/audit_results/



```
#!/bin/bash
mkdir -p /root/audit_results
find /usr -type f -perm /u=s -size -10M -exec cp -a {} /root/audit_results/ \;

chmod +x /usr/local/bin/audit_suid.sh
ls /root/audit_results/
```

■ cp -a preserva TODOS os atributos, inclusive o bit SUID. -size -10M: hífen = menor que. Sem sinal = exato. + = maior que

S5-Q29 — audit_suid.sh: salvar lista de SUID < 5MiB em arquivo



```
#!/bin/bash
find /usr -type f -perm /4000 -size -5M > /root/suid_files.txt

chmod +x /usr/local/bin/audit_suid.sh
cat /root/suid_files.txt
```

Flags do find — Referência:

Flag	Significado	Exemplo
-perm -4000	SUID ativo (pelo menos)	find /usr/bin -perm -4000
-perm /u=s	SUID ativo (forma simbólica)	find /usr -perm /u=s

<code>-perm /4000</code>	SUID ativo (alternativa)	<code>find /usr -perm /4000</code>
<code>-size -10M</code>	Menor que 10MB	<code>-size -5M</code> (menor que 5MB)
<code>-type f</code>	Apenas arquivos regulares	<code>-type d</code> (diretórios)
<code>-user root</code>	Dono é root	<code>-user alice</code>
<code>-exec CMD {} \;</code>	Executar CMD em cada resultado	<code>-exec cp -a {} /dest/ \;</code>

B9

grep · sed · Redirecionamento

^âncora · [class] · s/old/new/g · > sobrescreve · >> acrescenta · 2>&1

Referência rápida — grep, sed, redirecionamento:

```
# GREP — busca
grep "texto" arquivo           # busca simples
grep -i "texto" arquivo        # case-insensitive
grep -r "texto" /diretorio     # recursivo
grep "^Host" arquivo           # linhas que COMEÇAM com "Host"
grep -v "texto" arquivo        # linhas que NÃO contêm
grep '-0[56]-' arquivo         # character class [56] = 5 ou 6
grep -E '-(05|06)-' arquivo    # regex estendida (OR)

# SED — substituição
sed 's/antigo/novo/' arquivo   # 1ª ocorrência por linha
sed 's/antigo/novo/g' arquivo  # TODAS (g = global)
sed 's/sam/Sam/g' letter > newletter # resultado em novo arquivo

# REDIRECIONAMENTO
comando > arquivo              # SOBRESCREVE
comando >> arquivo              # ACRESCENTA
comando 2> arquivo             # só stderr
comando > arq 2>&1              # stdout + stderr
comando &> arquivo              # stdout + stderr (atalho Bash)
```

S1·Q25 — grep: linhas começando com 'Host' em sshd_config

```
# ^Host ancora no início da linha — exclui comentários automaticamente
grep -i '^Host' /etc/ssh/sshd_config > /root/ssh_hosts.txt
cat /root/ssh_hosts.txt
```

■ ^Host com ^ casa apenas linhas que COMEÇAM com Host. Comentários começam com # — não com Host — são exclu

S3·Q19 — sed: substituir 'sam' por 'Sam' em todas as ocorrências

```
echo 'sam is here. hello sam. sam again.' > letter

sed 's/sam/Sam/g' letter > newletter

cat newletter # Sam is here. hello Sam. Sam again.
cat letter    # sam is here. hello sam. sam again. (original intacto!)
```

■ Sem 'g': apenas 1ª ocorrência por linha. sed não modifica o arquivo original — > captura a saída.

S4·Q19 — redirecionar stdout E stderr para mesmo arquivo

```
ls /root /fake_directory > /var/tmp/ls_output.txt 2>&1
# OU forma curta Bash:
ls /root /fake_directory &> /var/tmp/ls_output.txt

cat /var/tmp/ls_output.txt # tem listagem E o erro
```

■ ORDEM IMPORTA: '> arquivo 2>&1' funciona. '2>&1 > arquivo' NÃO — stderr vai para o terminal.

S4·Q21 — grep com character class [56]: filtrar meses 05 e 06

```
cat > /root/dates.txt << EOF
2000-05-01 Sam
2002-06-15 Sue
2004-12-29 Sarah
2005-05-10 Mike
EOF

grep '-0[56]-' /root/dates.txt > /root/summer_birthdays.txt
# OU com regex estendida:
grep -E '-(05|06)-' /root/dates.txt > /root/summer_birthdays.txt

cat /root/summer_birthdays.txt # Sam (05), Sue (06), Mike (05)
```

■ [56] casa '5' ou '6'. Os hífen ao redor (-0[56]-) garantem que está no campo do mês — não confunde com ano (2005) ou

B10

Variáveis de Ambiente

export (sessão) · /etc/profile.d/ (todos) · /etc/environment (PAM+SSH)

S3·Q24 — export: variável na sessão atual e processos filhos

```
export EXAM_TYPE="RHCSA"

# Capturar saída de comando em variável
TODAY=$(date +%F) # ex: 2025-05-06

# Verificar em subshell (processo filho)
bash -c 'echo $EXAM_TYPE' # deve mostrar: RHCSA
echo $TODAY               # data atual
```

- export torna a variável disponível para processos filhos. Sem export, scripts rodados a partir desta sessão NÃO verão a

S4·Q20 — /etc/profile.d/: variável global para todos os usuários

```
vim /etc/profile.d/class_var.sh
# Conteúdo:
export CLASS="RHCSA"

# Verificar (simular login de outro usuário)
su - sarah
echo $CLASS # deve mostrar: RHCSA
```

- Qualquer arquivo .sh em /etc/profile.d/ é executado no login de TODOS. Mais limpo e seguro que editar /etc/profile direto

S5-Q24 — /etc/environment: variável global inclusive via SSH

```
vim /etc/environment
# Conteúdo (sem 'export', sem lógica shell):
VAR="RHCSA Prac Five"

# Verificar (logout e login, ou:
su -
echo $VAR    # RHCSA Prac Five
```

■ Não é script shell — não usa export, não suporta lógica. Lido pelo PAM para todos os tipos de login, inclusive SSH sem

Comparativo — quando usar cada abordagem:

Abordagem	Usa export?	Suporta lógica?	Funciona SSH?	Quando usar
export VAR=...	Sim	Sim	Não persiste	Sessão atual e filhos
/etc/profile.d/	Sim	Sim	Shell interativo	Variáveis/scripts globais com lógica
/etc/environment	Não	Não	Sim (PAM)	Variáveis estáticas para todos

Tabela de Referência — Comparadores Bash

Comparador	Significado	Exemplo
-eq	igual (números)	["\$A" -eq "\$B"]
-ne	diferente (números)	["\$A" -ne 0]
-gt	maior que	["\$NUM" -gt 0]
-lt	menor que	["\$NUM" -lt 10]
-ge	maior ou igual	["\$NUM" -ge 1]
-le	menor ou igual	["\$NUM" -le 100]
=	igual (strings)	["\$STR" = "sim"]
!=	diferente (strings)	["\$STR" != ""]
-f	é arquivo regular	[-f "/etc/passwd"]
-d	é diretório	[-d "/tmp"]
-z	string vazia	[-z "\$VAR"]
\$#	número de argumentos	["\$#" -ne 2]

■ Checklist — O que dominar ao final do Dia 06/05

■ Estrutura Base

- Todo script começa com `#!/bin/bash` e termina com `chmod +x`
- `$()` para capturar saída de comando, `$(( ))` para aritmética

■ Argumentos

- `flipargs.sh`: `echo "$2 $1"` (inverter argumentos)
- `checkfile.sh`: `if [ -f "$1" ] then/else` — aspas em `$1` obrigatórias

■ Loop for lista

- `user_audit.sh`: `for USER in lista + USERID=$(id -u $USER)`
- `$(id -u $USER)` retorna número — `$(id $USER)` retorna texto completo

■ for + \$@ + if

- `sum.sh`: `TOTAL=0` antes do loop + `for NUM in "$@" + if [ "$NUM" -gt 0 ]`
- `"$@"` com aspas preserva argumentos — `-gt` = greater than

■ while + arquivo

- `process_lines.sh`: `while read VAR1 VAR2 + done < arquivo`
- NÃO usar pipe (`cat | while`) — subshell perde variáveis

■ read + aritmética

- sum.sh interativo: `echo + read + $((num1 + num2))`
- Leia o enunciado: 'pede ao usuário'=read vs 'aceita argumentos'=\$1 \$2

■ case + tr

- yes-no.sh: `tr '[:upper:]' '[:lower:]'` antes do case
- Anatomia: `case/in/opção)/;; /*)/esac` — esquecer `;;` causa erro

■ find SUID

- find_suid.sh: `find /usr/bin -user root -perm -4000`
- `-perm -4000` (hífen = pelo menos). `-size -10M` (hífen = menor que)
- `-exec cp -a {} /dest/ \;` preserva atributos incluindo SUID

■ grep · sed · redir

- `grep -i '^Host'` arquivo > saída.txt (^ ancora no início)
- `sed 's/sam/Sam/g'` arquivo > novo (g=todas, original intacto)
- `> arquivo 2>&1` ou `&> arquivo` (ordem importa no `2>&1!`)
- `grep '[56]'` para character class — `grep -E '(05|06)'` para OR

■ Variáveis Ambiente

- `export VAR=valor` para sessão atual e processos filhos
- `/etc/profile.d/*.sh` para variável global com suporte a lógica
- `/etc/environment` para variável global sem export (funciona SSH/PAM)

RHCSA EX200 · Dia 06/05: Scripts Shell + grep + sed + find + Variáveis · Prova: 14/05 · Boa sorte!